Laporan Hands-on 2

Eksperimen End-to-End MLflow dan Scikit-learn

Nama Nobel Nizam
NIM 123450117
Tanggal 11 Oktober 2026

Abstract

Praktik ini membangun alur eksperimen klasifikasi tumor ganas dari persiapan data hingga rancangan pemuatan model produksi. Dataset Breast Cancer Wisconsin dibagi secara tetap, kemudian delapan konfigurasi baseline dan kandidat dibandingkan menggunakan 5-fold stratified cross-validation. SVC selanjutnya dituning pada 12 kombinasi parameter. Konfigurasi terbaik, yaitu $C = 10$ dan $\gamma =$ `scale`, memperoleh recall CV $0{,}965 \pm 0{,}029$ dengan precision $0{,}971 \pm 0{,}018$. Pada test set yang hanya digunakan sekali, model menghasilkan recall 0,929, precision 1,000, F1 0,963, dan ROC-AUC 0,993. Implementasi juga mencakup pencatatan MLflow, nested runs, aturan seleksi otomatis, Model Registry, quality gate, alias `champion`, DVC, dan audit lineage.

Contents

1 Pendahuluan **1**
1.1 Hipotesis eksperimen . . . 2

2 Lingkungan dan Rancangan Eksperimen **2**
2.1 Alur kerja . . . 2

3 Pelaksanaan dan Hasil **2**
3.1 L1 — Persiapan, Split, dan Versi Data . . . 2
3.2 L2 — Baseline dan Controlled Experiment . . . 3
3.3 L3 — Perbandingan Kandidat Model . . . 3
3.4 L4 — Hyperparameter Tuning dengan Nested Runs . . . 4
3.5 L5 — Seleksi Kandidat secara Programatik . . . 5
3.6 L6 — Evaluasi Final pada Test Set . . . 5
3.7 L7 — Model Registry dan Quality Gate . . . 5
3.8 L8 — Pemuatan Model dan Prediksi . . . 5
3.9 L9 — Reproducibility dan Audit Lineage . . . 6

4 Validasi Implementasi dan Keterbatasan **6**

5 Analisis Hipotesis **6**

6 Kesimpulan **7**

A Perintah Reproduksi **7**

1 Pendahuluan

Eksperimen machine learning tidak hanya menghasilkan sebuah model, tetapi juga melibatkan data, parameter, kode, lingkungan, metrik, dan artefak. Tanpa pencatatan yang konsisten, model terbaik sulit ditelusuri dan hasilnya sulit direproduksi. Hands-on 2 menerapkan alur end-to-end agar setiap keputusan model mempunyai dasar metrik dan lineage yang jelas.

Kasus yang digunakan adalah deteksi tumor ganas pada dataset Breast Cancer Wisconsin. Target asli dibalik sehingga kelas positif bernilai 1 untuk *malignant*. Kesalahan false negative dianggap lebih berbahaya karena tumor ganas dapat terlewat. Oleh sebab itu, recall menjadi *optimizing metric*, sedangkan precision minimal 0,90 menjadi *satisficing metric* agar alarm palsu tetap terkendali.

1.1 Hipotesis eksperimen

1. **H1:** model linear sederhana memberikan recall yang jauh lebih baik daripada baseline Dummy.
2. **H2:** standardisasi fitur meningkatkan recall model yang sensitif terhadap skala.
3. **H3:** tuning SVC meningkatkan recall tanpa menurunkan precision di bawah 0,90.

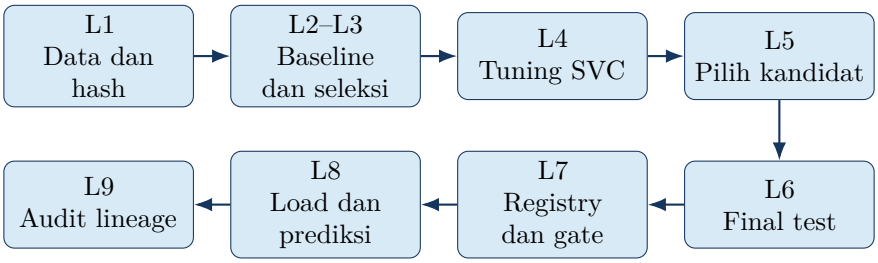
2 Lingkungan dan Rancangan Eksperimen

Eksperimen model dijalankan menggunakan Python 3.13.13, scikit-learn 1.9.0, pandas 3.0.5, dan NumPy 2.5.1. Semua model memakai split data dan fold cross-validation yang sama. Random seed ditetapkan ke 42 untuk mengurangi variasi yang tidak berasal dari model.

Table 1: Keputusan eksperimen sebelum training.

Aspek	Keputusan
Dataset	`sklearn.datasets.load_breast_cancer`, 569 sampel dan 30 fitur numerik
Kelas positif	1 = malignant setelah target asli dibalik
Split	80% train dan 20% test, stratified, seed 42
Seleksi model	5-fold Stratified CV pada train set
Optimizing metric	Recall
Satisficing metric	Precision $\geq$ 0,90
Test set	Digunakan sekali setelah kandidat dipilih

2.1 Alur kerja



Kode dipisahkan berdasarkan tanggung jawab: `data.py` menangani data, `tracking.py` menyeragamkan logging, `train.py` menjalankan eksperimen, `evaluate.py` memilih dan menguji kandidat, sedangkan `register.py`, `predict.py`, dan `reproduce.py` menangani siklus model setelah training.

3 Pelaksanaan dan Hasil

3.1 L1 — Persiapan, Split, dan Versi Data

Dataset dimuat sebagai DataFrame, target dibalik, lalu dibagi secara stratified. Train set disimpan sebagai `data/train.csv` dan test set sebagai `data/test.csv`. Sidik jari dataset dihitung dari

seluruh nilai dan indeks menggunakan MD5. Pipeline DVC didefinisikan pada `dvc.yaml`; file CSV diabaikan Git sehingga versi data direpresentasikan oleh metadata DVC.

```
python -m src.data --force
# Train: 455 baris
# Test : 114 baris
# Data MD5: ee5481ad12c0896439e993aacc9e8d50
```

Table 2: Distribusi kelas setelah split.

Subset	Benign (0)	Malignant (1)	Total
Train	285	170	455
Test	72	42	114

Hasil. Split menghasilkan proporsi kelas yang hampir sama pada kedua subset. Hash yang tetap memberikan identitas data yang dapat dicatat pada setiap run dan diperiksa kembali saat reproduksi.

3.2 L2 — Baseline dan Controlled Experiment

Baseline Dummy selalu memprediksi kelas mayoritas. Dua Logistic Regression kemudian dibandingkan dengan satu perubahan saja, yaitu penggunaan `StandardScaler`. Semua model menggunakan data dan fold yang sama.

Table 3: Hasil baseline (rata-rata ± simpangan baku CV).

Model	Recall	Precision	F1
Dummy	$0{,}000 \pm 0{,}000$	$0{,}000 \pm 0{,}000$	$0{,}000 \pm 0{,}000$
LogReg tanpa scaling	$0{,}918 \pm 0{,}034$	$0{,}958 \pm 0{,}024$	$0{,}937 \pm 0{,}018$
LogReg + scaling	$0{,}953 \pm 0{,}040$	$0{,}977 \pm 0{,}028$	$0{,}964 \pm 0{,}021$

Hasil. Dummy gagal mendeteksi seluruh tumor ganas. Logistic Regression tanpa scaling langsung mencapai recall 0,918. Standardisasi menaikkan recall sekitar 0,035 dan sekaligus meningkatkan precision serta F1. H1 dan H2 mendapat dukungan awal.

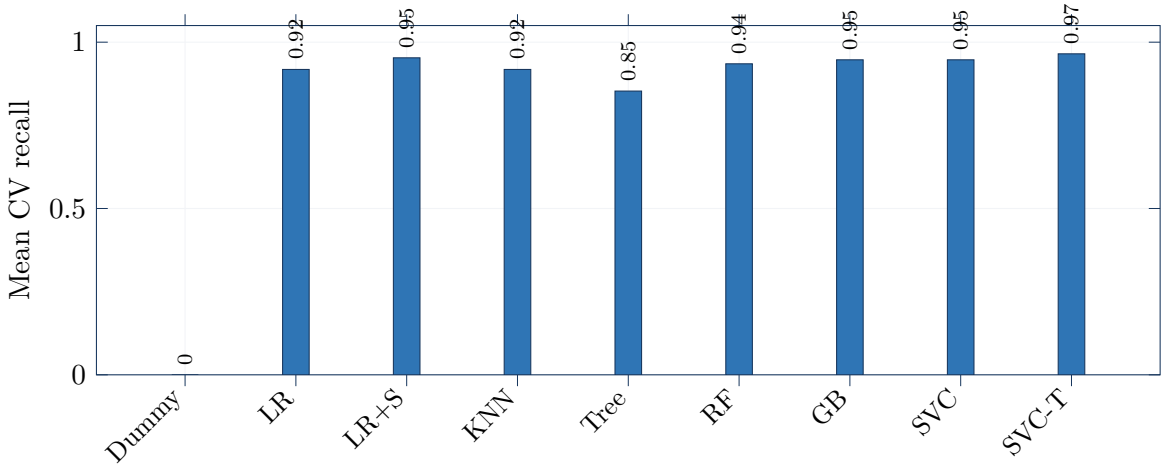
3.3 L3 — Perbandingan Kandidat Model

Lima algoritma tambahan dibandingkan: KNN, Decision Tree, Random Forest, Gradient Boosting, dan SVC. Preprocessing serta fold CV dibuat identik agar perbedaan metrik terutama berasal dari algoritma.

Decision Tree menjadi satu-satunya model non-Dummy yang tidak memenuhi precision 0,90. Logistic Regression dengan scaling menghasilkan recall tertinggi sebelum tuning, sedangkan SVC mempunyai kombinasi recall, precision, F1, dan ROC-AUC yang kuat sehingga layak dituning.

Table 4: Ringkasan seluruh hasil model sebelum tuning.

Model	Recall	Precision	F1	ROC-AUC
Dummy	$0{,}000 \pm 0{,}000$	$0{,}000 \pm 0{,}000$	$0{,}000 \pm 0{,}000$	$0{,}500 \pm 0{,}000$
LogReg tanpa scaling	$0{,}918 \pm 0{,}034$	$0{,}958 \pm 0{,}024$	$0{,}937 \pm 0{,}018$	$0{,}993 \pm 0{,}006$
LogReg + scaling	$0{,}953 \pm 0{,}040$	$0{,}977 \pm 0{,}028$	$0{,}964 \pm 0{,}021$	$0{,}996 \pm 0{,}005$
KNN	$0{,}918 \pm 0{,}034$	$0{,}988 \pm 0{,}025$	$0{,}951 \pm 0{,}026$	$0{,}987 \pm 0{,}014$
Decision Tree	$0{,}853 \pm 0{,}085$	$0{,}894 \pm 0{,}037$	$0{,}872 \pm 0{,}057$	$0{,}887 \pm 0{,}061$
Random Forest	$0{,}935 \pm 0{,}039$	$0{,}964 \pm 0{,}011$	$0{,}949 \pm 0{,}021$	$0{,}989 \pm 0{,}007$
Gradient Boosting	$0{,}947 \pm 0{,}034$	$0{,}965 \pm 0{,}021$	$0{,}955 \pm 0{,}020$	$0{,}991 \pm 0{,}005$
SVC default	$0{,}947 \pm 0{,}034$	$0{,}977 \pm 0{,}021$	$0{,}961 \pm 0{,}008$	$0{,}995 \pm 0{,}005$



3.4 L4 — Hyperparameter Tuning dengan Nested Runs

SVC RBF dituning memakai `GridSearchCV`. Ruang pencarian berisi empat nilai C dan tiga nilai γ, sehingga terdapat 12 kombinasi. Implementasi MLflow membuat satu parent run `10-svc-gridsearch`; setiap kombinasi dicatat sebagai child run beserta parameter, mean, dan standard deviation seluruh metrik.

Table 5: Konfigurasi dan hasil terbaik tuning SVC.

Komponen	Hasil
Ruang C	0,1; 1; 10; 100
Ruang γ	`scale`; 0,01; 0,001
Jumlah kombinasi	12
Parameter terbaik	$C = 10$, $\gamma =$ `scale`
CV recall	$0{,}965 \pm 0{,}029$
CV precision	$0{,}971 \pm 0{,}018$
CV F1	$0{,}967 \pm 0{,}011$
CV ROC-AUC	$0{,}993 \pm 0{,}006$

Hasil. Tuning meningkatkan point estimate recall SVC dari 0,947 menjadi 0,965 tanpa melanggar precision minimum. Kenaikan 0,018 masih lebih kecil daripada simpangan baku recall antarfold, sehingga hasil ini belum cukup untuk klaim peningkatan yang kuat secara statistik.

3.5 L5 — Seleksi Kandidat secara Programatik

Run kandidat difilter dengan aturan `cv_precision_mean >= 0.90`, lalu diurutkan berdasarkan recall dan F1. Aturan ini menghindari pemilihan manual berdasarkan perasaan atau hanya satu metrik. Dengan hasil yang diperoleh, parent run SVC hasil tuning menjadi kandidat utama karena recall CV tertinggi dan precision 0,971.

Hasil. Kandidat terpilih adalah SVC dengan $C = 10$ dan $\gamma =$ `scale`. Decision Tree otomatis tersaring karena precision rata-ratanya hanya 0,894.

3.6 L6 — Evaluasi Final pada Test Set

Model kandidat dilatih ulang pada seluruh train set, kemudian test set digunakan satu kali. Skrip `evaluate.py` menolak evaluasi ulang bila run sudah memiliki tag `test_evaluated=true`. Metrik test ditambahkan pada run kandidat yang sama agar lineage tidak terputus.

Table 6: Hasil final test set untuk SVC hasil tuning.

Metrik	Nilai
Recall	0,9286
Precision	1,0000
F1	0,9630
ROC-AUC	0,9927

Confusion matrix

	Pred. benign	Pred. malignant
Aktual benign	72	0
Aktual malignant	3	39

Hasil. Tidak terdapat false positive, tetapi tiga dari 42 tumor ganas terlewat. Recall test turun sekitar 0,036 dari mean CV, sedangkan precision tetap sempurna. Model masih memenuhi quality gate precision.

3.7 L7 — Model Registry dan Quality Gate

Skrip `register.py` mendaftarkan URI kandidat sebagai versi baru pada registry model. Setelah itu, skrip menambahkan alias `challenger` dan membandingkannya dengan champion. Promosi dilakukan jika test precision minimal 0,90 dan test recall setidaknya 0,005 di atas champion. Hash data champion dan challenger juga harus sama agar perbandingan adil.

```
passed = (test_precision >= 0.90 and
          test_recall >= champion_recall + 0.005)
```

Aturan quality gate telah diuji dengan unit test untuk kondisi lolos, precision terlalu rendah, dan peningkatan recall yang belum cukup. Pada registry kosong, kandidat pertama dengan precision 1,000 memenuhi syarat menjadi champion awal.

3.8 L8 — Pemuatan Model dan Prediksi

Aplikasi tidak menggunakan path file atau nomor versi yang ditulis langsung. Skrip `predict.py` memuat model melalui URI berikut:

```
models:/bc-malignant-classifier@champion
```

Pendekatan alias memisahkan kode aplikasi dari versi model. Ketika champion berubah, kode prediksi tetap sama; service hanya perlu memuat ulang model. Signature yang disimpan bersama model membantu memeriksa struktur dan tipe input.

3.9 L9 — Reproducibility dan Audit Lineage

Skrip `reproduce.py` menelusuri alias champion menuju model version dan run sumber. Run tersebut menyediakan nama run, parameter, metrik, hash data, serta commit Git. Hash data saat ini dibandingkan dengan tag `data_md5`; ketidaksesuaian menghentikan proses karena hasil tidak lagi dapat dianggap reproduksi dari eksperimen yang sama.

```
Prediksi -> @champion -> model version -> run
         -> parameter, metrik, commit Git, data MD5, artifact
```

Commit implementasi yang menjadi dasar eksperimen adalah `98bc3e1`, sedangkan hash dataset adalah `ee5481ad12c0896439e993aacc9e8d50`.

4 Validasi Implementasi dan Keterbatasan

Validasi yang berhasil dilakukan mencakup kompilasi seluruh modul Python, empat unit test untuk split, hash, dan quality gate, serta eksekusi scikit-learn terhadap baseline, seluruh kandidat, 12 kombinasi tuning, dan final test. Angka hasil pada laporan berasal dari eksekusi tersebut.

Table 7: Status validasi praktik.

Komponen	Status	Keterangan
Split, hash, dan CSV	Berhasil	Data train/test dibuat dan hash stabil
Baseline serta model selection	Berhasil	Delapan konfigurasi dieksekusi dengan 5-fold CV
GridSearchCV	Berhasil	Seluruh 12 kombinasi selesai
Final test	Berhasil	Metrik dan confusion matrix dihitung
Unit test dan compile check	Berhasil	Empat unit test lulus; modul dapat dikompilasi
MLflow server dan UI	Belum dijalankan	Paket MLflow tidak tersedia dan instalasi jaringan diblokir
DVC runtime	Belum dijalankan	Metadata DVC tersedia, tetapi paket DVC tidak tersedia
Registry, alias, dan serving	Belum dijalankan	Kode tersedia; memerlukan server MLflow aktif

Keterbatasan tersebut berarti laporan tidak mengklaim adanya run, screenshot MLflow UI, model version, atau alias yang benar-benar tersimpan pada server. Setelah dependensi dapat dipasang, alur yang sama dapat dijalankan melalui `run_pipeline.sh` untuk menghasilkan artefak tracking dan registry.

5 Analisis Hipotesis

1. **H1 didukung.** Recall meningkat dari 0,000 pada Dummy menjadi 0,918 pada Logistic Regression tanpa scaling. Baseline Dummy menunjukkan bahwa accuracy kelas mayoritas tidak cukup untuk kasus medis yang berfokus pada kelas positif.
2. **H2 didukung pada eksperimen ini.** StandardScaler menaikkan recall Logistic Regression dari 0,918 menjadi 0,953 dan F1 dari 0,937 menjadi 0,964. Perbandingan terkontrol membuat scaling menjadi satu-satunya perubahan utama.
3. **H3 didukung secara point estimate, tetapi belum kuat secara statistik.** Tuning menaikkan recall SVC dari 0,947 menjadi 0,965 dengan precision 0,971. Selisih recall 0,018

masih berada dalam variasi antarfold, sehingga repeated CV atau pengujian statistik diperlukan untuk klaim yang lebih kuat.

Hasil test memperlihatkan trade-off yang sesuai tujuan: tidak ada benign yang salah diberi alarm malignant, tetapi masih ada tiga malignant yang terlewat. Dalam konteks klinis, threshold keputusan dapat ditinjau kembali untuk mengejar recall lebih tinggi, kemudian dampaknya terhadap precision harus diukur pada data validasi yang terpisah.

6 Kesimpulan

Hands-on 2 berhasil menunjukkan bahwa eksperimen model yang baik membutuhkan lebih dari sekadar nilai akurasi. Split dan fold yang tetap membuat perbandingan adil; baseline membuktikan manfaat model; scaling meningkatkan model linear; dan tuning SVC menghasilkan kandidat dengan recall CV tertinggi tanpa melanggar precision minimum. Pada test set, model terbaik mencapai recall 0,929, precision 1,000, F1 0,963, dan ROC-AUC 0,993.

Praktik juga memperlihatkan peran MLflow dan DVC dalam rancangan MLOps: parameter, data, metrik, model, dan lineage ditempatkan dalam satu alur; seleksi kandidat serta quality gate dinyatakan sebagai aturan kode; dan aplikasi memuat model melalui alias, bukan versi yang ditulis langsung. Dengan demikian, eksperimen menjadi lebih terukur, dapat diaudit, dan siap dihubungkan ke otomasi CI/CD/CT setelah tracking server serta dependensi tersedia.

A Perintah Reproduksi

```
cd handson-2
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
dvc repro prepare

# Terminal 1
./start_server.sh

# Terminal 2
./run_pipeline.sh
```

Tahap juga dapat dijalankan terpisah dalam urutan berikut:

```
src.data -> src.train -> src.evaluate -> src.register
         -> src.predict -> src.reproduce
```

MLflow UI tersedia pada http://127.0.0.1:5000 ketika server aktif.